

Problem Set 2.1 (Bonus) – Climatic Risk Intelligence Module

Kshitiz Reddy
M.Tech AIML, School of AI, Bennett University

Source Code

```
1 """
2 Program: Climatic Risk Intelligence Module
3 Purpose: Classifies a work-site into a safety tier (Freeze Alert, Critical,
4          Cautionary, Operational) from a derived Heat Stress Index, using
5          the walrus operator for input capture and short-circuit logic.
6 Author: Kshitiz
7
8 Heuristic Tiering Model: HSI = Temperature + (0.5 * Humidity). Freeze
9 Alert is checked first regardless of HSI, since a high-humidity freezing
10 site can still produce a numerically large HSI. Critical and Cautionary
11 are checked next in that order; Operational is the catch-all else, which
12 avoids an unnecessary explicit lower-bound condition.
13 """
14
15 if (temp_str := input("Temperature (C): ").strip()):
16     try:
17         temperature = float(temp_str)
18     except ValueError:
19         temperature = None
20 else:
21     temperature = None
22
23 if temperature is None:
24     print("Safety Level: Unknown")
25 else:
26     humidity_str = input("Humidity (%): ").strip()
27     try:
28         humidity = float(humidity_str)
29         assert humidity >= 0, "Telemetry Error: Negative Humidity"
30     except ValueError:
31         humidity = None
32
33     if humidity is None:
34         print("Safety Level: Unknown")
35     else:
36         hsi = temperature + (0.5 * humidity)
37
38         if temperature <= 0:
39             tier = "FREEZE ALERT"
40         elif hsi > 45 or (temperature > 38 and humidity > 70):
41             tier = "CRITICAL"
42         elif 30 <= hsi <= 45:
43             wind_str = input("Wind speed (km/h): ").strip()
44             wind_speed = float(wind_str) if wind_str else 0.0
45             tier = "CAUTIONARY" if wind_speed < 5 else "OPERATIONAL"
46         else:
47             tier = "OPERATIONAL"
48
49     # Challenge Task: Nested Intelligence Suite (battery adjustment)
50     if tier == "CAUTIONARY":
51         battery_str = input("Battery level (%): ").strip()
```

```

52     battery = float(battery_str) if battery_str else 100.0
53     if battery < 20:
54         tier = "CRITICAL"
55     elif battery > 80:
56         tier = "OPERATIONAL"
57
58     risk_label = "Safe" if hsi < 30 else "Unsafe"
59     print(f"HSI: {hsi:.1f}")
60     print(f"Safety Level: {tier}")
61     print(f"Risk Label: {risk_label}")

```

Execution Trace

```

1  --- Scenario (a): Empty input handled by walrus check ---
2  Temperature (C):
3  Safety Level: Unknown
4
5  --- Scenario (b): Critical heatwave reading ---
6  Temperature (C): 32
7  Humidity (%): 80
8  HSI: 72.0
9  Safety Level: CRITICAL
10 Risk Label: Unsafe
11
12 --- Scenario (c): Cautionary reading adjusted by Battery-Level bonus ---
13 Temperature (C): 25
14 Humidity (%): 20
15 Wind speed (km/h): 3
16 Battery level (%): 10
17 HSI: 35.0
18 Safety Level: CRITICAL
19 Risk Label: Unsafe
20
21 Note: primary tier for (25, 20, wind 3) is CAUTIONARY (30 <= HSI <= 45,
22 wind < 5). Battery level 10 is below the 20% threshold, so the Nested
23 Intelligence Suite elevates it to CRITICAL.

```

Submission Notes

Freeze Alert is checked before anything else because temperature alone decides it – a freezing site with high humidity can still produce a large HSI, and checking HSI-based tiers first would misclassify it as a heat emergency. Critical and Cautionary come next in that order, so by the time a Cautionary branch runs it already knows HSI is not above 45.

The empty-input check runs on the raw string from `input()`, guarded by the walrus operator, before any conversion happens. This matters because a temperature of 0 is falsy as a number but not falsy as the string "0", so checking the raw string avoids treating a genuine 0°C reading as missing input.